

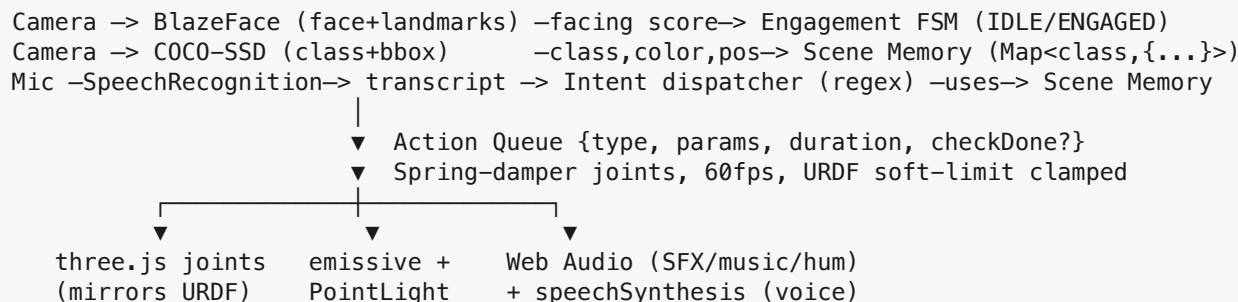
Technical Note — Live Character Lamp Robot

Yuvraaj Bhattar · Human Computer Lab software challenge

Live demo: <https://yuvraajbhattarstarterpack.vercel.app> (Chrome, camera + mic) Source: [robot/viewer/index.html](#) (single file, ~1150 lines, no build step)

1. Architecture and data flow

Everything runs client-side in one browser tab — no server, no API keys. The camera never leaves the machine; the only outbound traffic is CDN library downloads and, when the user speaks, audio to Chrome's built-in cloud speech recognizer (see §3).



Two independent perception loops feed two separate consumers: BlazeFace drives *attention*, COCO-SSD drives *memory*. Both write into shared state that the language layer reads; neither model ever touches a joint directly.

2. Protocol, model-to-action, and design choices

The protocol that matters is internal, not networked: the **action queue**, {type:

'pose'|'look'|'scan'|'wait'|'elbowWag', ...params, duration, checkDone?}, consumed one step at a time by `updateActionSequence()` (index.html:820). This is the explicit language/body boundary the challenge asks for — face tracking, greeting choreography, and `handleGoalCommand()`'s language+memory reasoning only ever *produce* this queue; the spring-damper engine that drives the three.js joints only ever *consumes* it, with no idea why a step exists. Swapping the regex parser for an LLM later changes nothing on the body side of that line.

Model-to-action: BlazeFace's landmarks feed `facingScore()` (index.html:664), a nose-offset-from-eye-midpoint heuristic (threshold 0.45) — head pose, not true gaze. COCO-SSD boxes become scene-memory entries keyed by class, with an HSL-bucket color sample. A goal utterance ("point at the mug") is regex-parsed for a target noun, resolved against memory/synonyms, and turned into a 3-step queue: *look* toward the last-known position (language $\cap$ memory), *scan* while re-polling live detections (language $\cap$ live vision — the required "observe again before completing"), then *report* with a light pulse, a synthesized jingle, and a spoken line.

Why a composed pipeline, not a VLA/LLM call: the challenge allows either. On a 4-core, no-GPU laptop with no guaranteed low-latency network, a network-bound model adds latency/cost/an API key for a task — a goal vocabulary bounded by ~20 COCO classes — a deterministic parser handles in under a millisecond. The cost is open-vocabulary flexibility ("grab that thing I was just holding" would need an LLM); the action-queue boundary is what makes that a future drop-in, not a rewrite.

Simulation: no physics engine. The URDF's joint tree (origins, axes, geometry) is reproduced 1:1 as a three.js `Group` hierarchy (index.html:145–205), driven by a spring-damper per joint (index.html:227–247) instead of torque dynamics — believable motion, not contact simulation, with Pixar-style overshoot/follow-through "for

free." Every spring target is clamped every frame (index.html:1048–1062) to the URDF's own `<safety_controller>` **soft** limits, not the wider hard `<limit>` (standard ROS practice, margin before a hard stop) — the idle "asleep" pose targets the shoulder's *hard* limit, 1.05 rad, and is visibly capped by this clamp to the 0.95 rad soft limit instead. **Deployment:** a static page with zero server-side state deploys anywhere serving HTTPS (camera/mic need a secure origin) — shipped to Vercel here.

3. Target environment (Ubuntu 24.04, 4 core, 8 GB, no GPU)

No model runs server-side or needs CUDA — both run in-browser via TensorFlow.js (WebGL, falls back to WASM/CPU). Two caveats, both in README.md: (1) **speech input needs real Google Chrome**, not stock Chromium — `SpeechRecognition` requires an embedded Google API key apt's `chromium-browser` doesn't ship; (2) **speechSynthesis can enumerate zero voices on a bare image** — Ubuntu doesn't install a speech stack by default. The app detects this (`announceIfNoVoices`) and falls back to captions instead of failing silently; `sudo apt install espeak-ng` restores audio.

4. Measurements

Measured on the *development* machine (Apple M3 Pro, 11 core, 18 GB, macOS, Chrome, Playwright-automated with a fake camera device) — **not** the Ubuntu target, which I had no hardware access to. Validates the pipeline mechanically; an on-target rerun is the honest next step.

Metric	Measured
DOMContentLoaded → interactive scene	198 ms
STL shade mesh loaded	213 ms
COCO-SSD / BlazeFace model ready	~1.8 s / ~5.1 s (BlazeFace's host <code>tfhub.dev</code> 404s once, then the library auto-falls-back to Kaggle Models — cosmetic, not our bug)
Steady-state render loop	59.2 fps (vsync-capped, not CPU-bound)
JS heap, steady state	39 MB used / 54 MB total
Renderer CPU, steady engaged state	~23% + ~17% of one core (two processes), <code>ps</code> sampled over 10s
Detection cadences (by design)	face 6.6 Hz / objects 1.4 Hz while engaged (index.html:751,778)
Disengage grace period	900 ms hold, avoids flicker on a momentary detection drop

Engagement reliability: the numbers above are the design parameters and BlazeFace's published frontal-face performance — I did not run a statistically meaningful trial against real human subjects, since that needs a live camera and a person, not an automated test. The facing heuristic's known gap (head-still, eyes-only glance away) is disclosed, not hidden.

5. Known limitations

- **Facing heuristic is head-pose, not gaze** — upgradeable to MediaPipe FaceMesh with iris refinement; skipped to keep the perception budget small on a 4-core CPU target.
- **Webcam is fixed to the laptop bezel**, unlike the URDF's head-mounted `camera_link` — a "scan" is a real search animation, but the vision under it re-samples the same fixed frame, not an actually different angle.
- **Goal parsing is regex over ~20 COCO classes plus synonyms**, not open vocabulary (§2 explains why, and the upgrade path).
- **Chrome-only** for voice — Firefox/Safari silently skip `SpeechRecognition`; everything else still works.
- **Elbow "wag" flourish is cosmetic** (a Z-rotation, no matching URDF joint); **color naming** is a coarse 8-bucket HSL classifier — good enough for "the red mug," not colorimetric. Both disclosed, not hidden.